Categorical Variables

02/23/2026

Today's Plan

1. Logistics/Schedule Update

2. Recap & ANOVA

Logistics / Schedule Updates

- Thank you for your patience 🙏🏽
- Few Schedule Updates:
  - **Mon Mar 2:** Project Proposal Deadline (extended)
    - We'll have the template repo up by **Wed Feb 25** so you can submit early if you're ready!
  - **2 HWs left:**
    - HW 3: Available this Wed, **due Midnight Monday**, review Tuesday
    - HW 4: Available Wed Mar 4, **due Midnight Mon Mar 9**, review Tues Mar 10

Logistics / Schedule Updates

- Thank you for your patience 🙏🏽
- Few Schedule Updates:
  - **Today:** Short refresher, Notebook Review, Time to start readings
  - **Tomorrow:** Discussion on readings + new notebook(s)
  - **Wed:** Notebook(s) Review & Clarifications so you're set for HW 3

Questions?

Recap

How does the GLM see different variables?

Continuous Variables

Categorical Variables

Design Matrix: The “eyes” of the GLM

$$y = X\beta + \epsilon$$

- **Categorical Variables**
  - We can **encode** them into numbers using a variety of approaches
  - A variable with **k unique levels** is represented with **k-1 columns** in the Design Matrix

Parameter Interpretation

- Whenever we have more than 1 predictor in a GLM **estimates assume other predictors = 0**

$$\text{balance}_i = \beta_0 + \beta_1 \text{student\_dummy}_i + \beta_2 \text{income}_i$$

Slope of income
when student_dummy = 0

Parameter Interpretation

- Whenever we have more than 1 predictor in a GLM **estimates assume other predictors = 0**

$$\text{balance}_i = \beta_0 + \beta_1 \text{student\_dummy}_i + \beta_2 \text{income}_i$$

Slope of student_dummy
when income = 0

- This is rarely ever what we want, unless 0 is a meaningful value for the predictor!
- But there's a simple solution: **centering predictors**
  - If we center a predictor(s) then estimates will assume **predictor(s) = centered value (typically mean)**

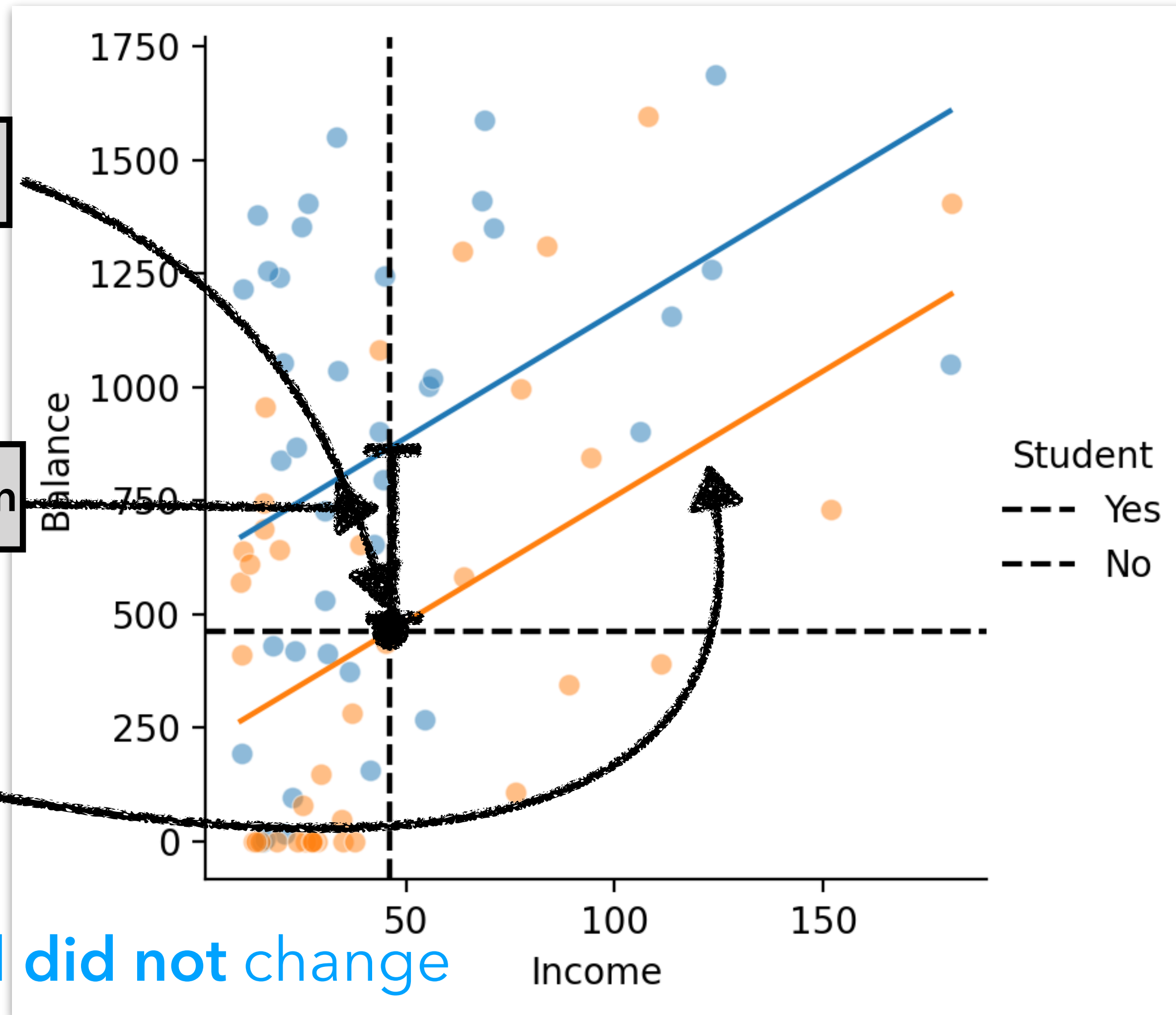
Model A: 2 intercepts; 1 slope **centered income**

$$\text{balance}_i = \beta_0 + \beta_1 \text{student\_dummy}_i + \beta_2 \text{income}_i$$

$\hat{\beta}_0$ = **Student = 0 (No); Income = mean**

$\hat{\beta}_1$ = **Student (Yes - No); Income = mean**

$\hat{\beta}_2$ = **Income Slope; Student = 0 (No)**



predictor we centered **did not** change

Parameter Interpretation

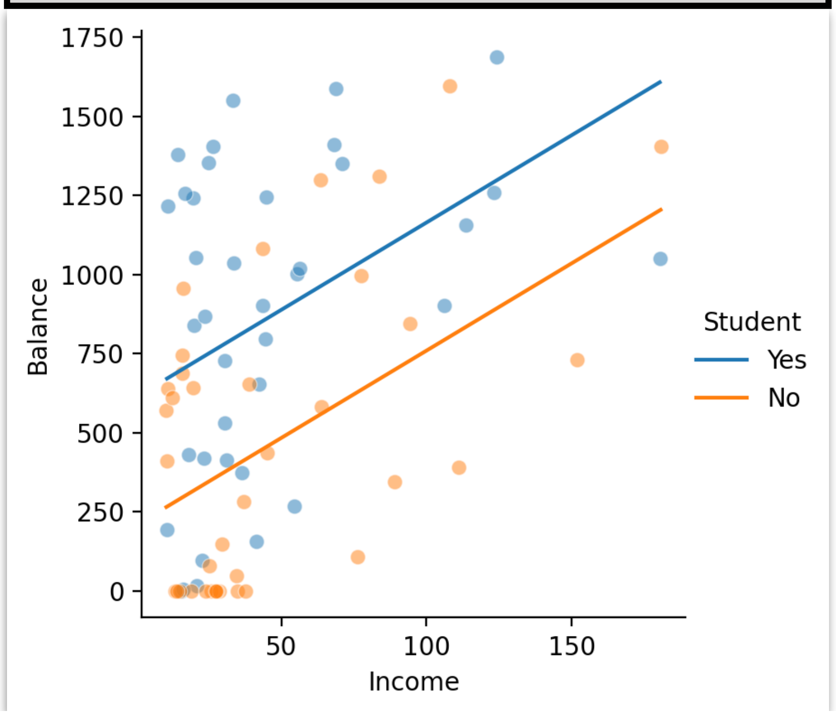
$$\text{balance}_i = \beta_0 + \beta_1 \text{student\_dummy}_i + \beta_2 \text{income}_i$$

Original

$\hat{\beta}_0$ = Student = 0 (No); Income = 0

$\hat{\beta}_1$ = Student (Yes - No); Income = 0

$\hat{\beta}_2$ = Income Slope; Student = 0 (No)

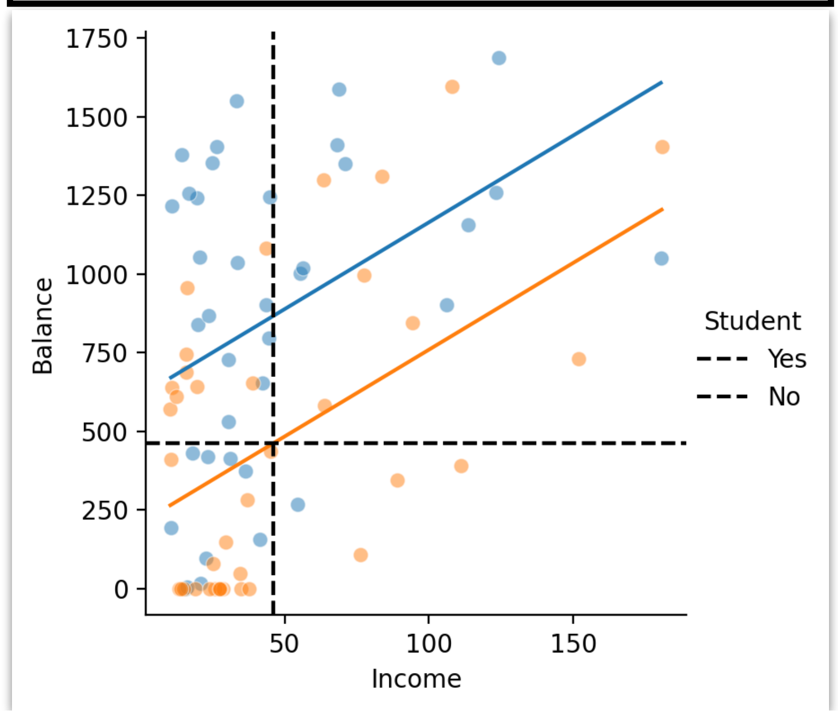


Centered

$\hat{\beta}_0$ = Student = 0 (No); Income = mean

$\hat{\beta}_1$ = Student (Yes - No); Income = mean

$\hat{\beta}_2$ = Income Slope; Student = 0 (No)



Parameter Interpretation Summary

$$\text{balance}_i = \beta_0 + \beta_1 \text{student\_dummy}_i + \beta_2 \text{income}_i$$

- Whenever we have more than 1 predictor in a GLM **estimates assume other predictors = 0**
- This is rarely ever what we want, unless 0 is a meaningful value for the predictor!
- But there's a simple solution: **centering predictors**
    - If we center a predictor(s) then estimates will assume **predictor(s) = centered value (typically mean)**
- **Does NOT** change estimate of centered variable
- **Does NOT** change model predictions
- **Rule of Thumb:**
    - Whenever you have *any* categorical predictors, center your continuous predictors (it's almost always what you actually want to estimate)

Categorical predictors (3+ levels)

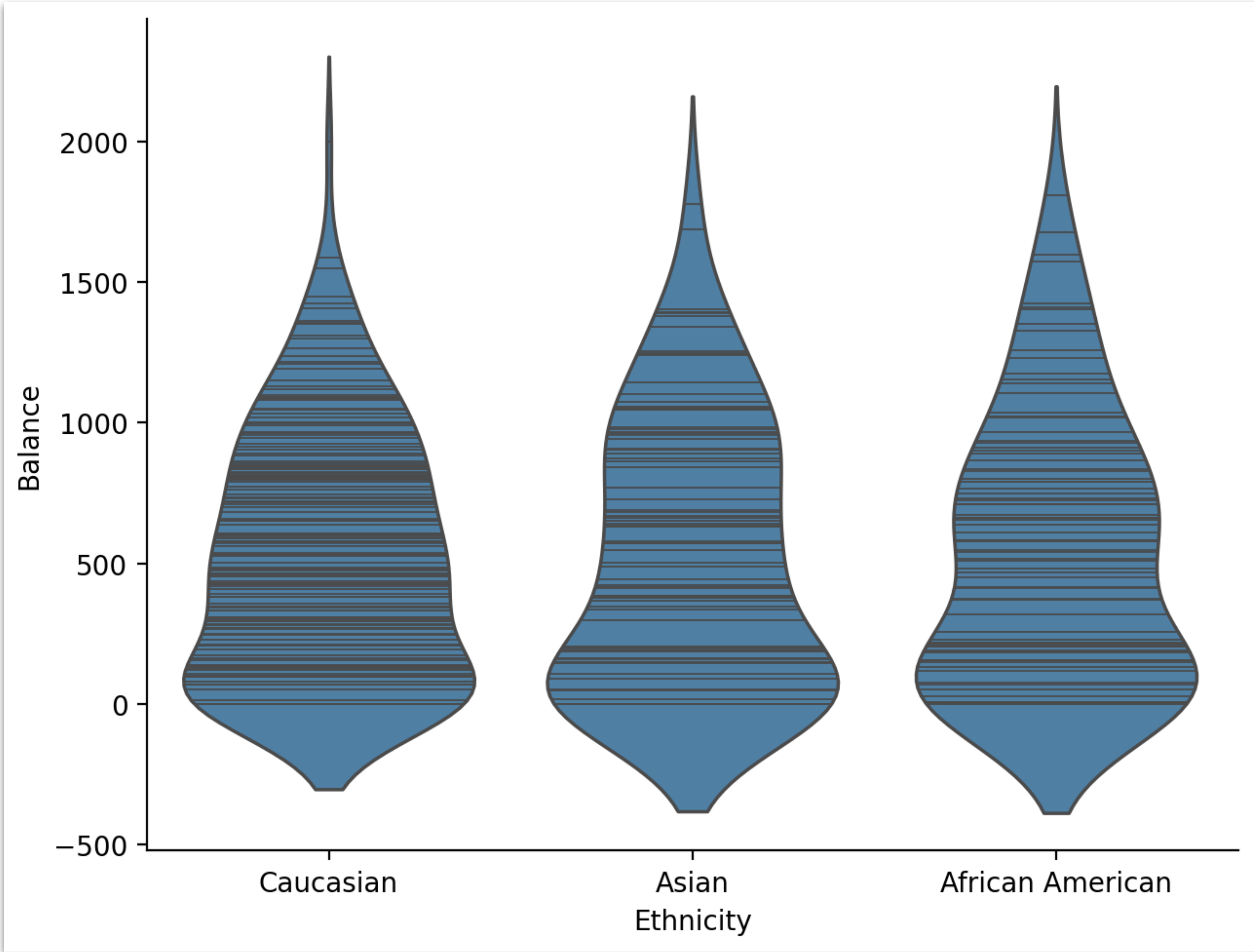
Dataset: Credit-Card Debt

index	income	limit	rating	cards	age	education	gender	student	married	ethnicity	balance
1	14.89	3606	283	2	34	11	Male	No	Yes	Caucasian	333
2	106.03	6645	483	3	82	15	Female	Yes	Yes	Asian	903
3	104.59	7075	514	4	71	11	Male	No	No	Asian	580
4	148.92	9504	681	3	36	11	Female	No	No	Asian	964
5	55.88	4897	357	2	68	16	Male	No	Yes	Caucasian	331
6	80.18	8047	569	4	77	10	Male	No	No	Caucasian	1151
7	21.00	3388	259	2	37	12	Female	No	No	African American	203
8	71.41	7114	512	2	87	9	Male	No	No	Asian	872
9	15.12	3300	266	5	66	13	Female	No	No	Caucasian	279
10	71.06	6819	491	3	41	19	Female	Yes	Yes	African American	1350

variable	description
income	in thousand dollars
limit	credit limit
rating	credit rating
cards	number of credit cards
age	in years
education	years of education
gender	male or female
student	student or not
married	married or not
ethnicity	African American, Asian, Caucasian
balance	average credit card debt in dollars

Does ethnicity predict credit card balance?

Visualize the data first



H_0: Ethnicity does not predict balance

H_1: Ethnicity does predict balance

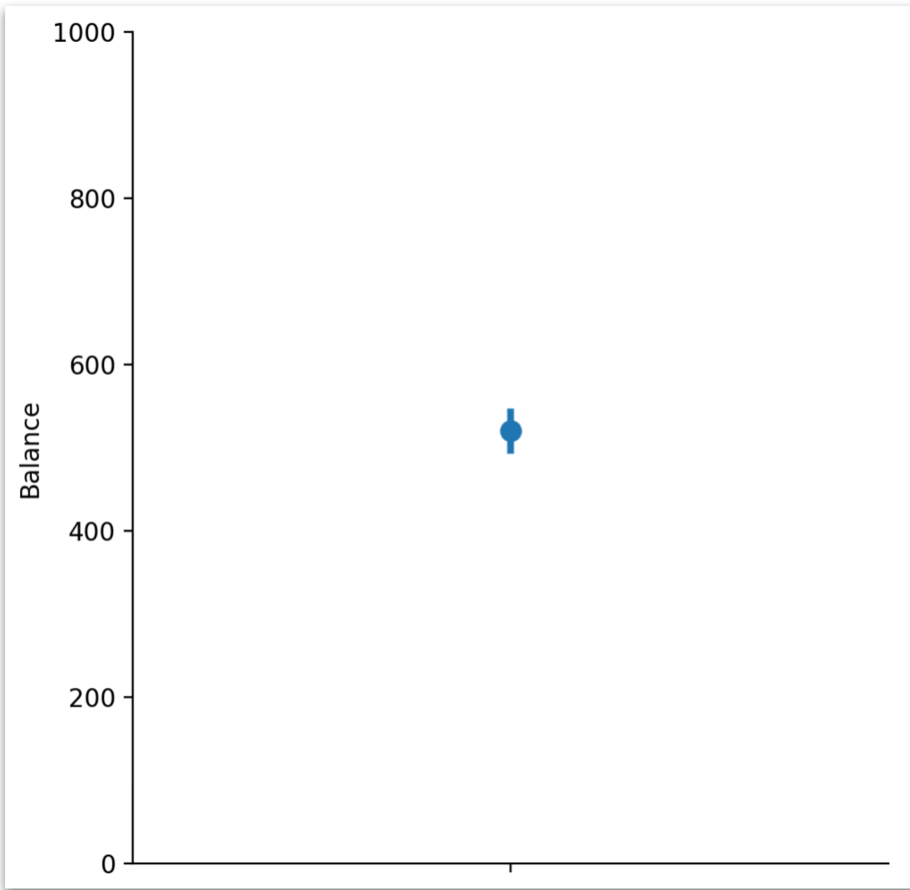
Model C

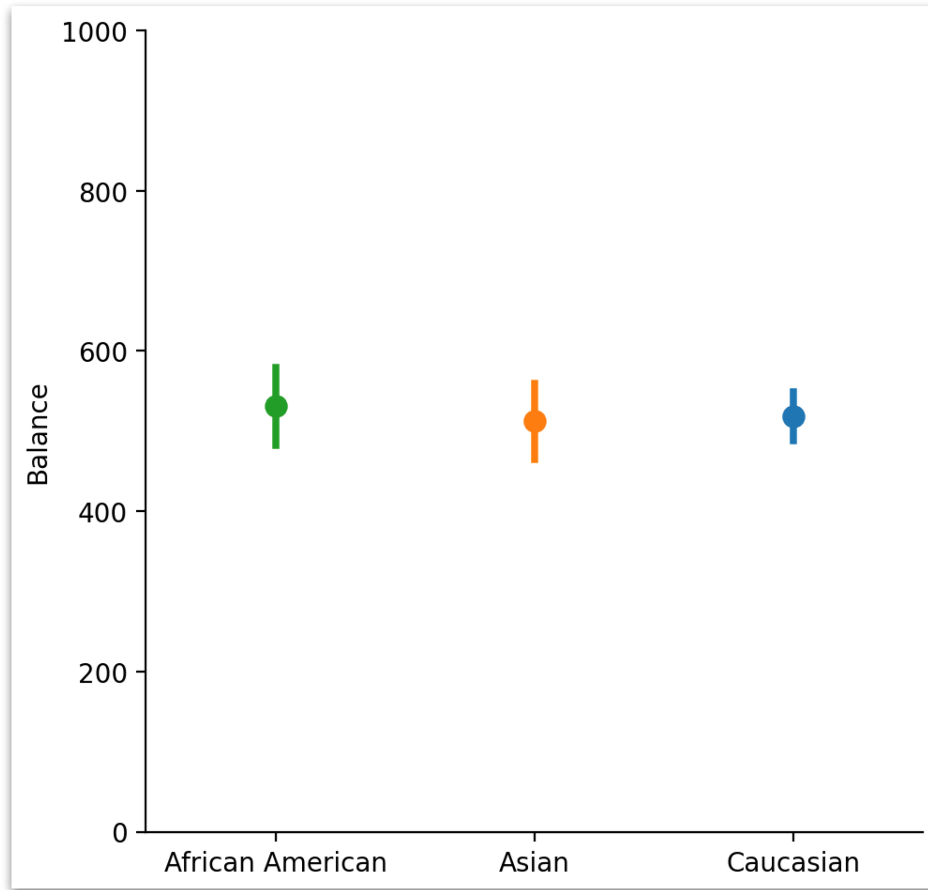
$$Y_i = \beta_0 + \epsilon_i$$

Model A

$$Y_i = \beta_0 + \beta_1 X_i + \epsilon_i$$

ethnicity





Worth it?

```python
# Imports
from bossanova import model, compare

# Compact Model
model_c = model("Balance ~ 1", credit)

# Augmented Model
model_a = model("Balance ~ Ethnicity", credit)

# Compare
compare(model_c, model_a)
```

model str	PRE f64	F f64	rss f64	ss f64	df f64	df_resid f64	p_value f64
Balance ~ 1			84,339,911.91			399	
Balance ~ Ethnicity	0	0.043	84,321,457.71	18,454.2	2	397	0.957

Not Worth it!

You just did a "One-way ANOVA"!

```python
# Imports
from bossanova import model, compare

# Compact Model
model_c = model("Balance ~ 1", credit)

# Augmented Model
model_a = model("Balance ~ Ethnicity", credit)

# Compare
compare(model_c, model_a)
```

model str	PRE f64	F f64	rss f64	ss f64	df f64	df_resid f64	p_value f64
Balance ~ 1			84,339,911.91			399	
Balance ~ Ethnicity	0	0.043	84,321,457.71	18,454.2	2	397	0.957

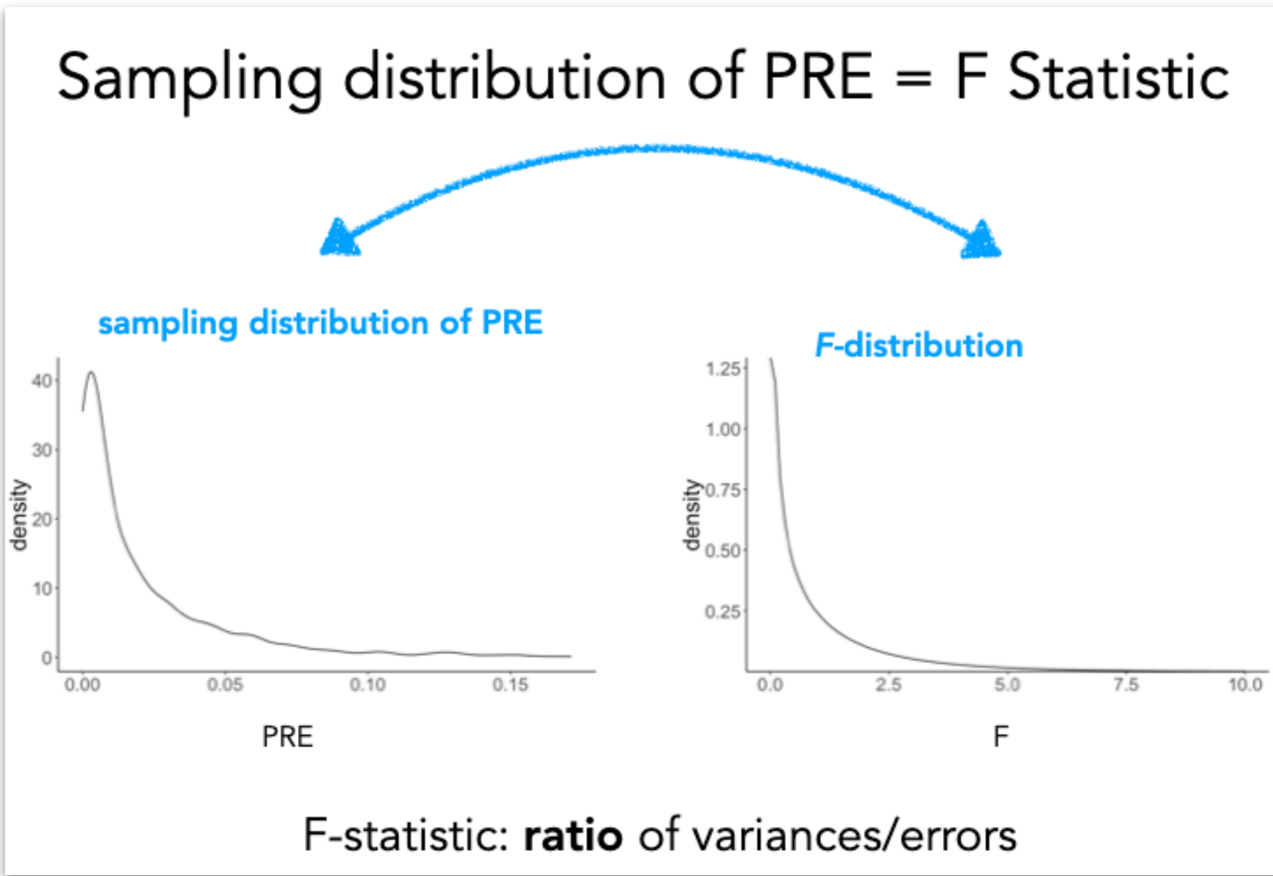
Same Thing!

```python
# Joint-test (ANOVA)
model_a.jointtest()
```

df1 i64	df2 f64	f_ratio f64	p_value f64
2	397	0.043442783049415894	0.957491

ANOVA = Nested Model Comparison

- Analysis-of-Variance: any time we use categorical variable(s) in a GLM
- Developed by Ronald Fisher (**F**-distribution)



ANOVA = Nested Model Comparison

- Analysis-of-Variance: any time we use categorical variable(s) in a GLM
- Developed by Ronald Fisher (**F**-distribution)
- Mathematical trick to calculate **unique variance** (model error) attributable to each factor

```
m_int = model("Balance ~ Ethnicity * Student", credit)
```

Factor 1: **Ethnicity**

Factor 2: **Student**

	AA	A	Ca
Yes			
No			

ANOVA = Nested Model Comparison

- Analysis-of-Variance: any time we use categorical variable(s) in a GLM
- Developed by Ronald Fisher (**F**-distribution)
- Mathematical trick to calculate **unique variance** (model error) attributable to each factor

```
m_int = model("Balance ~ Ethnicity * Student", credit)
```

term str	df1 i64	df2 f64	f_ratio f64	p_value f64
Ethnicity	2	394	0.39578765341267563	0.673416
Student	1	394	11.252490509838259	0.000872
Ethnicity:Student	2	394	1.5134138803469321	0.221434

ANOVA = Nested Model Comparison

```
m_int = model("Balance ~ Ethnicity * Student", credit)
```

term str	df1 i64	df2 f64	f_ratio f64	p_value f64
Ethnicity	2	394	0.39578765341267563	0.673416
Student	1	394	11.252490509838259	0.000872
Ethnicity:Student	2	394	1.5134138803469321	0.221434

```
# Drop Interaction
m_add = model("Balance ~ Ethnicity + Student").fit()

# Drop Ethnicity (keep interaction!)
drop_eth = model("Balance ~ Student + Ethnicity:Student", credit).fit()

# Drop Student (keep interaction!)
drop_student = model("Balance ~ Ethnicity + Ethnicity:Student", credit).fit()
```

```
# Interaction Effect
compare(m_add, m_int)
```

model str	PRE f64	F f64
Balance ~ Ethnicity + Student		
Balance ~ Ethnicity * Student	0.0076	1.5134

```
# Main effect of Student
compare(drop_student, m_int)
```

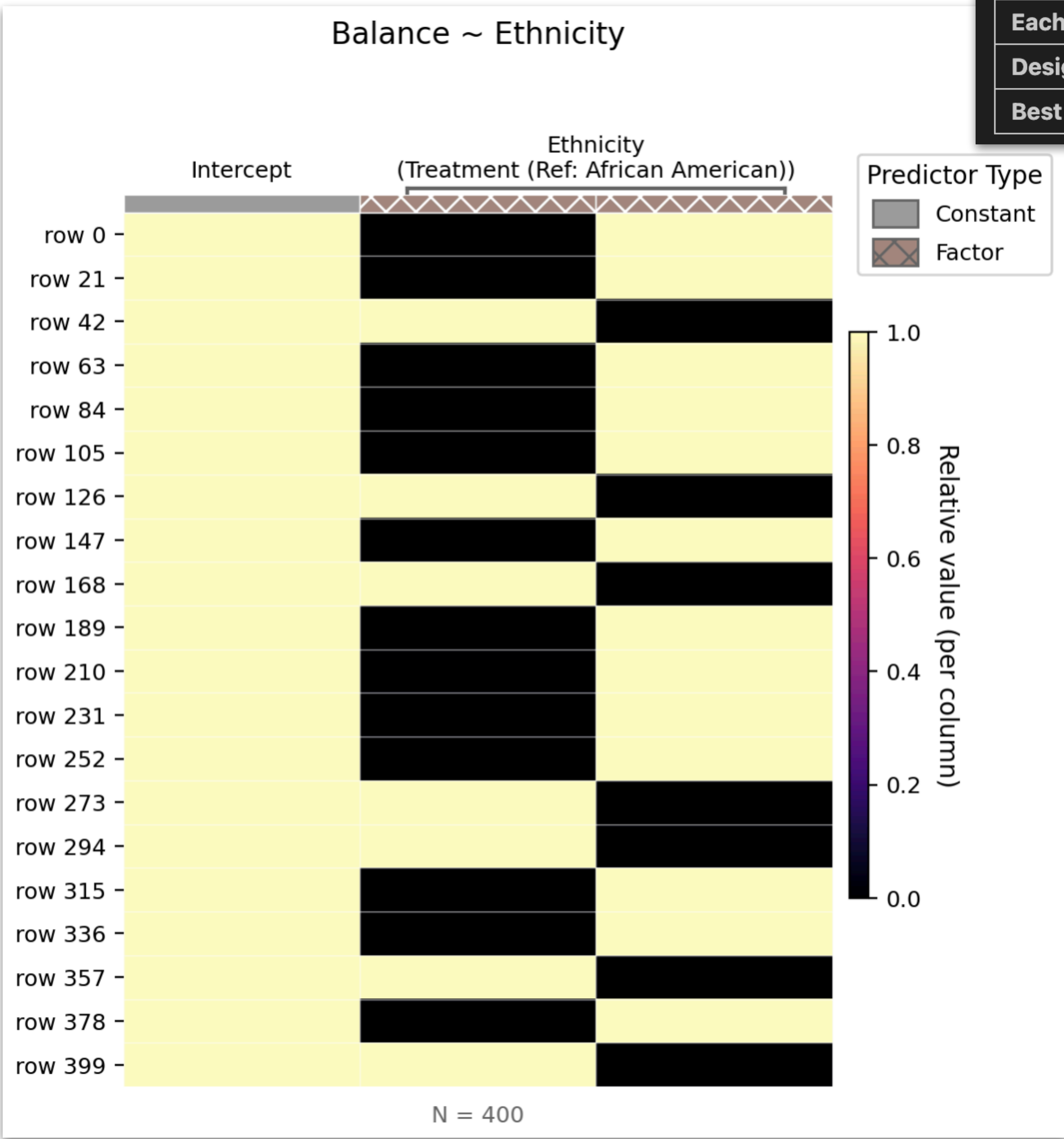
model str	PRE f64	F f64
Balance ~ Ethnicity + Ethnicity:Student		
Balance ~ Ethnicity * Student	0.0278	11.2525

```
# Main effect of Ethnicity
compare(drop_eth, m_int)
```

model str	PRE f64	F f64
Balance ~ Student + Ethnicity:Student		
Balance ~ Ethnicity * Student	0.002	0.3958

Encoding Scheme 1: Treatment (Dummy)

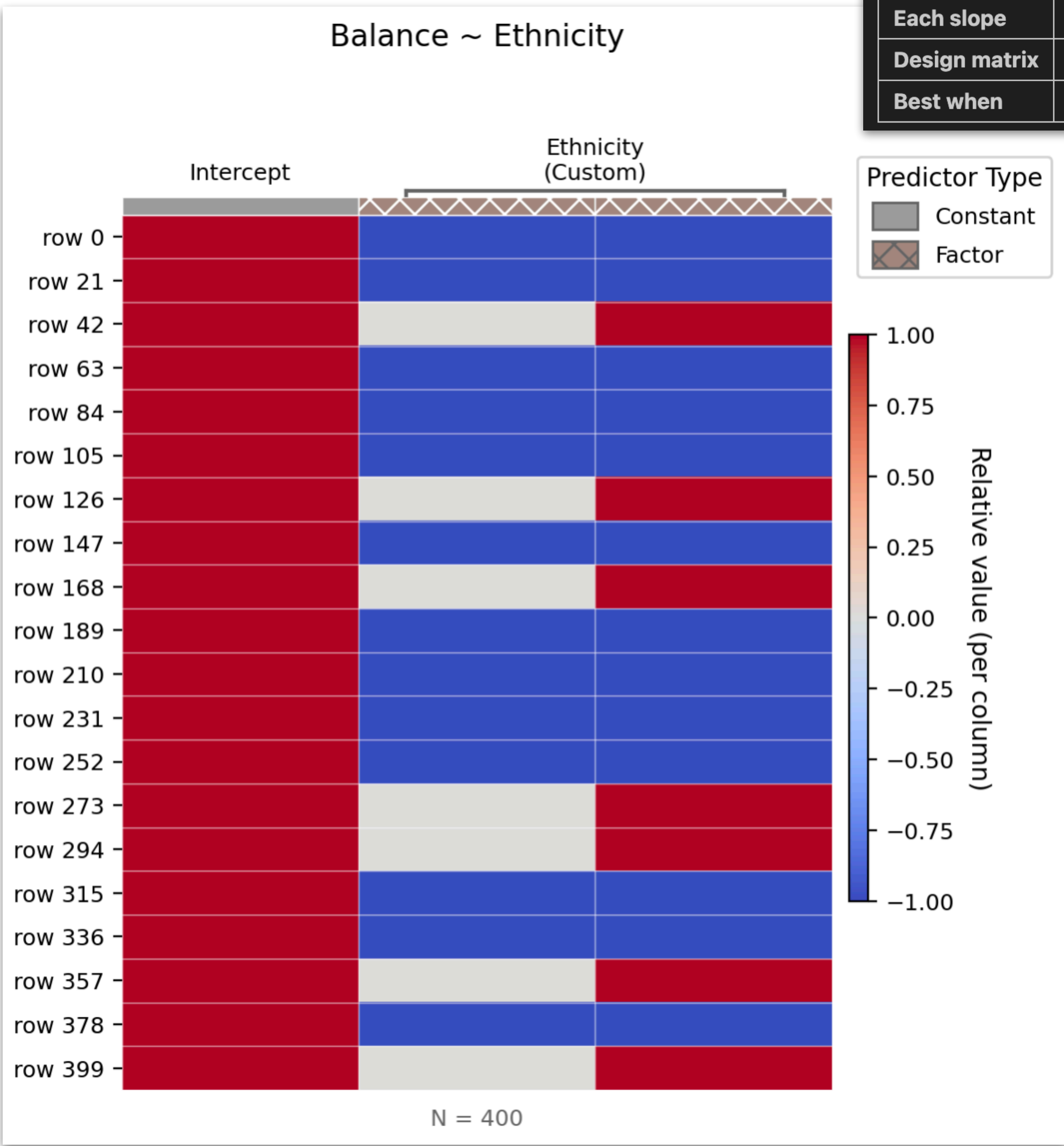
Property	Value
Intercept	Mean of the reference group
Each slope	Difference from reference group
Design matrix	0/1 pattern (reference = all zeros)
Best when	There's a natural reference group (e.g., control condition)



- Default in Python and R
- Reference group:
  - bossanova = first **alphabetically**
  - R = first based on appearance
- **Will not** yield valid F-tests (ANOVA)!

Encoding Scheme 2: Sum (Deviation)

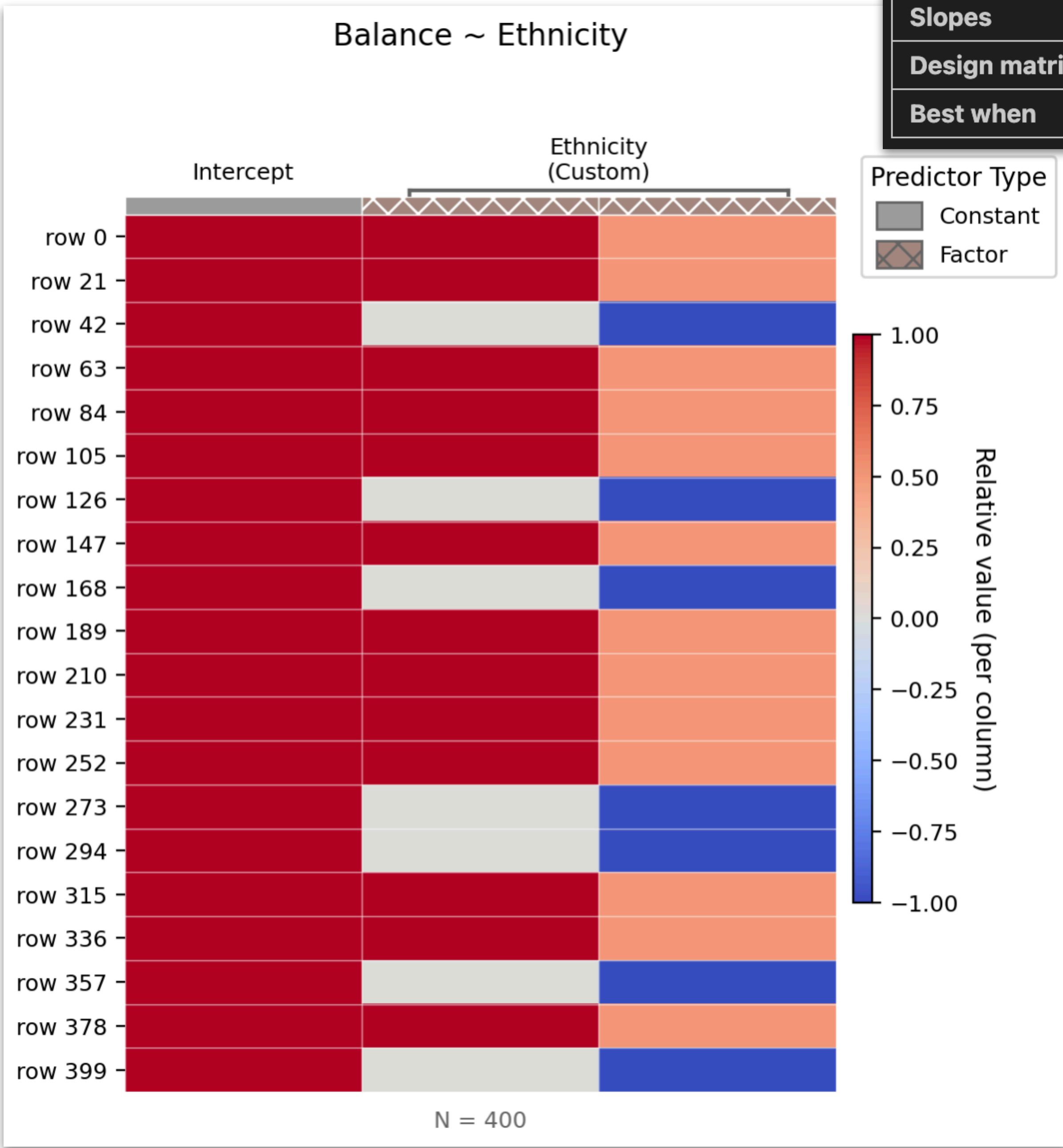
Property	Value
Intercept	Grand mean (unweighted mean of group means)
Each slope	Deviation of that group from the grand mean
Design matrix	-1 / 0 / 1 pattern (last group = all -1s)
Best when	No natural reference; you want each group compared to the overall average



- Each level is encoded as 1 and last level is encoded as -1
- Intercept = **grand mean**
- Slopes = **deviations from grand-mean**
- **Does** yield valid F-tests (ANOVA)

Encoding Scheme 3: Polynomial

Property	Value
Intercept	Grand mean
Slopes	Linear trend, quadratic trend, etc.
Design matrix	Orthogonal polynomial contrasts
Best when	Levels have a natural ordering (e.g., Low / Medium / High)



- Intercept = **grand mean**
- Each slope is an independent **trend** over factor levels (e.g. linear, quadratic, etc)
- **Does** yield valid F-tests (ANOVA)

ANOVA Summary

- Analysis-of-Variance: any time we use categorical variable(s) in a GLM
- Developed by Ronald Fisher (**F**-distribution)
- Mathematical trick to calculate **unique variance** (model **error**) attributable to each factor
- **Note**: Using model.jointtest() in Python (or jointtest from emmeans in R) will **always use the right encoding scheme**
    - This is called the "**Type III** Sum-of-Squares" approach

ANOVA Summary

This is the standard in the literature

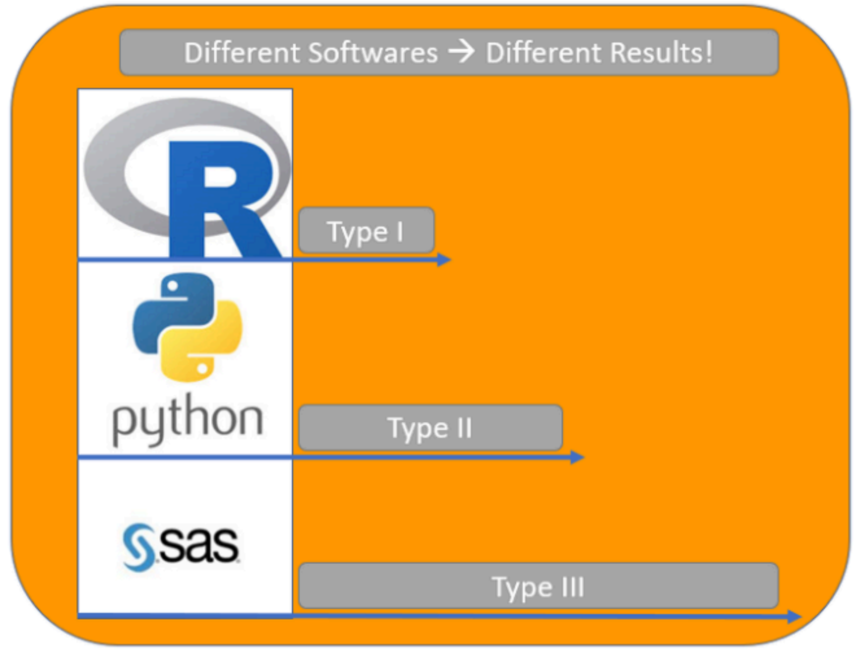
The Type III Sums of Squares are also called partial sums of squares again another way of computing Sums of Squares:

- Like Type II, the Type III Sums of Squares are not sequential, so the order of specification does not matter.
- Unlike Type II, the Type III Sums of Squares do specify an interaction effect.

Sums of Squares are Mathematically defined as:

- SS(A | B, AB) for independent variable A
- SS(B | A, AB) for independent variable B

But be careful of software defaults!



Default Types of Sums of Squares for different programming languages

not great for reproducibility ...

Recommendation: if you primarily care about ANOVA style analyses stick `jointtest` using `bossanova` in Python or `emmeans` in R. *Otherwise be careful to always use Sum or Polynomial encoding before fitting your model or use **model comparison!***

For the rest of today...

Start on the readings for tomorrow's discussion or come see us if you have additional questions!